2 академических часа · Для начинающих

Лекция №2. Управление Docker-контейнерами

Запуск, остановка, команды и веб-сервер Nginx

Что мы уже знаем

- Docker установлен в Debian 13.
- Образ — шаблон с файлами приложения.
- Контейнер — экземпляр, созданный из образа.
- В первой лабораторной мы запустили hello-world.

Что изучим сегодня

- Научимся запускать и называть контейнеры.
- Посмотрим, какие контейнеры работают.
- Научимся останавливать и запускать их снова.
- Выполним команды внутри контейнера.
- Запустим сайт Nginx и откроем его в браузере.

Учебные вопросы

1	**Шаг 1**	1. Образ, контейнер и жизненный цикл.
2	**Шаг 2**	2. Команда docker run и её параметры.
3	**Шаг 3**	3. Просмотр, остановка, запуск и удаление.
4	**Шаг 4**	4. Работа в интерактивном и фоновом режиме.
5	**Шаг 5**	5. Nginx, порты, логи и диагностика.

Образ и контейнер

Что именно мы запускаем

Образ — это ещё не запущенная программа

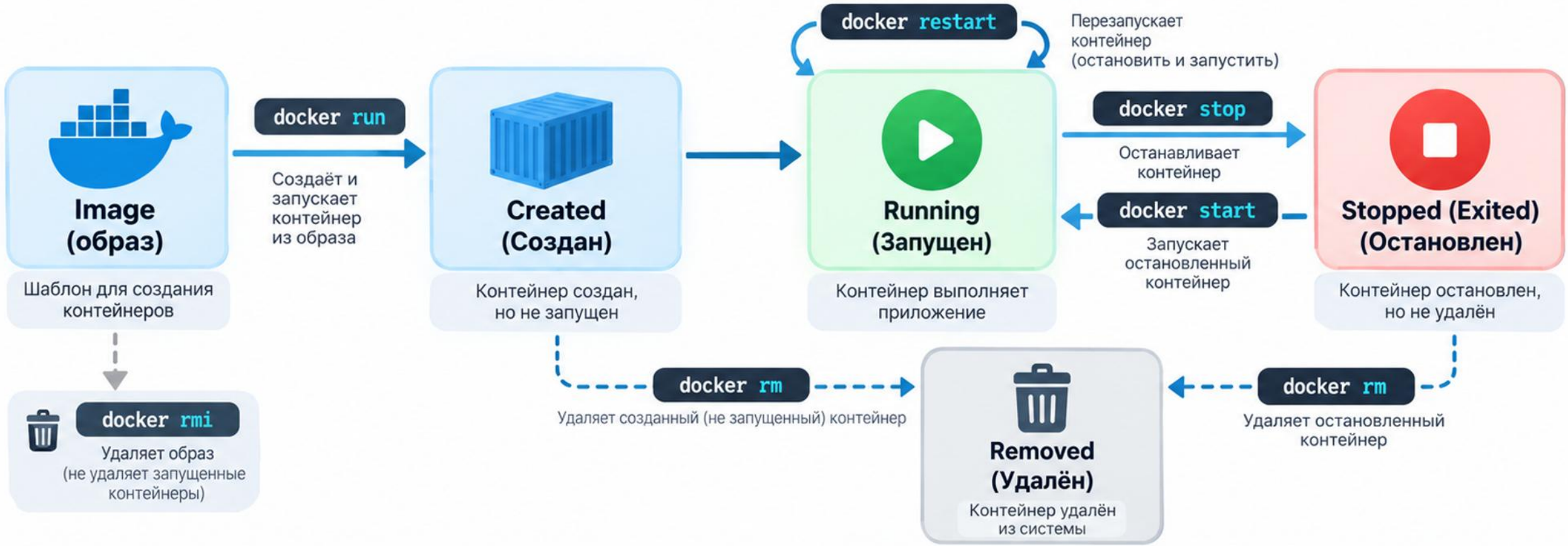
- Образ nginx:stable содержит файлы для Nginx.
- После загрузки образ хранится на компьютере.
- Чтобы Nginx заработал, создаём из образа контейнер.
- Из одного образа можно создать несколько контейнеров.

Контейнер может менять состояние

- Created — контейнер создан.
- Running — приложение работает.
- Exited — приложение завершилось или контейнер остановлен.
- Удалённый контейнер исчезает из списка, но образ обычно остаётся.

Жизненный цикл контейнера

Основные состояния Docker-контейнера и команды управления



Важно помнить

- Контейнер создается из образа.
- После stop контейнер не удаляется.
- Один образ может создать много контейнеров.
- Удаление контейнера не удаляет образ автоматически.

Что обычно видит студент

`docker ps`	— только запущенные контейнеры
`docker ps -a`	— все контейнеры
`docker logs <имя>`	— журналы контейнера
`docker exec -it <имя> /bin/sh`	— вход внутрь контейнера

Посмотреть образы

```bash
sudo docker images
```

Что важно

- Показывает скачанные образы.
- IMAGE — название образа, TAG — его версия или метка.

Посмотреть работающие контейнеры

```bash
sudo docker ps
```

Что важно

- Выводит только запущенные контейнеры.
- Пока контейнеров нет, список может быть пустым.

Посмотреть все контейнеры

BASH

```
sudo docker ps -a
```

Что важно

- -a означает all — все.
- Здесь видны и остановленные контейнеры.

Разница между образом и контейнером

Образ	Контейнер
Шаблон для запуска	Запущенный или остановленный экземпляр
docker images	docker ps -a
Может использоваться многократно	Имеет имя и собственное состояние

Создаём контейнер

Команда docker run

Самая простая команда запуска

```bash
sudo docker run hello-world
```

Что важно

- Docker находит образ hello-world.
- Создаёт контейнер, запускает его и показывает сообщение.
- Поскольку программа сразу завершается, контейнер переходит в Exited.

Запускаем Nginx

```bash
sudo docker run -d --name web1 -p 127.0.0.1:8080:80 nginx:stable
```

Что важно

- -d запускает в фоновом режиме.
- --name web1 задаёт имя контейнера.
- -p подключает порт Debian 8080 к порту контейнера 80.
- 127.0.0.1 ограничивает доступ самим Debian-хостом.

Разберём команду по частям

- sudo — выполняем с правами администратора.
- docker run — создаём и запускаем контейнер.
- -d — терминал остаётся свободным.
- --name web1 — понятное имя вместо случайного.
- nginx:stable — образ и его метка.

Что такое порт?

- Веб-сервер слушает сетевой порт — например, 80.
- Чтобы открыть страницу с Debian, используем порт 8080.
- В записи 8080:80 слева порт Debian, справа порт контейнера.
- 127.0.0.1 — доступ только с самого Debian.

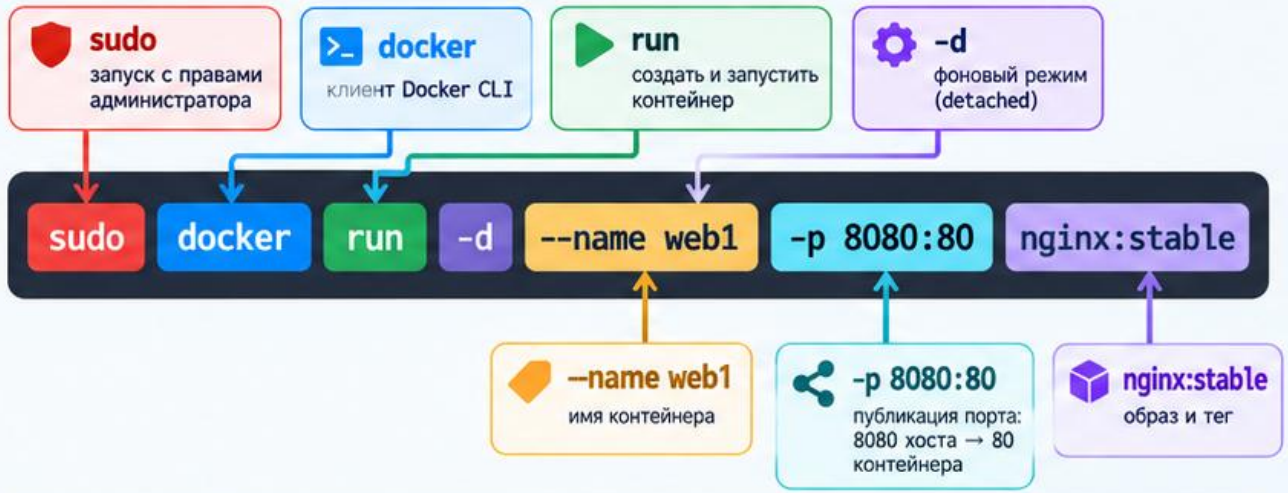
Анатомия docker run

Как одна команда создаёт и запускает контейнер

Что делает docker run

1. Проверяет, есть ли образ локально.
2. При необходимости скачивает образ из реестра.
3. Создаёт контейнер из образа.
4. Применяет параметры запуска.
5. Запускает основной процесс контейнера.

Разбор команды по частям



Как это работает под капотом



Часто используемые параметры

Параметр	Описание
-d	запуск в фоне
-it	интерактивный терминал
--name	задать имя контейнера
-p	проброс портов
-e	переменные окружения
-v	тома и каталоги
--rm	удалить контейнер после завершения
--restart	политика перезапуска

Что получаем в результате

- **Контейнер создан** — Из образа nginx:stable
- **Контейнер запущен** — Работает в фоновом режиме (-d)
- **Сервис доступен на порту 8080** — Порт 8080 хоста → 80 контейнера
- **Проверка:** `http://localhost:8080`

Важно помнить

1. Образ — шаблон, контейнер — его экземпляр
2. `docker run` объединяет `create` + `start`
3. Если основной процесс завершится, контейнер остановится

Публикация порта: localhost:8080 → контейнер Nginx:80



Команда запуска

```
sudo docker run -d --name web1 \
  -p 127.0.0.1:8080:80 nginx:stable
```

Как это работает

1. Слева указан порт хоста: **8080**
2. Справа указан порт контейнера: **80**
3. Запрос к localhost:8080 попадает в Nginx внутри контейнера

Важно знать

localhost в Debian — это сама Debian.

Запрос отправляется на текущий сервер (хост), а не во внешний интернет.

Проверяем сайт Nginx

```bash
curl http://localhost:8080
```

Что важно

- curl отправляет HTTP-запрос к Nginx.
- Если сервер доступен, получим HTML стартовой страницы.
- Вместо curl можно открыть адрес в браузере Debian.

Почему в браузере Windows может не открываться?

- localhost в Debian — это сама Debian.
- localhost в Windows — это Windows, а не виртуальная машина.
- В этой лекции проверяем Nginx внутри Debian.
- Сетевой доступ с Windows рассмотрим отдельно.

Управляем контейнером

Остановка, повторный запуск и удаление

Остановить контейнер

BASH

```
sudo docker stop web1
```

Что важно

- Останавливает основной процесс контейнера.
- Контейнер остаётся в системе и будет виден в docker ps -a.

Запустить остановленный контейнер

```bash
sudo docker start web1
```

Что важно

- Запускает уже существующий контейнер.
- При этом новый контейнер не создаётся.

Перезапустить контейнер

```bash
sudo docker restart web1
```

Что важно

- Останавливает и запускает тот же контейнер.
- Может пригодиться после изменения настроек приложения.

Удалить контейнер

BASH

```
sudo docker stop web1
sudo docker rm web1
```

Что важно

- Останавливаем контейнер, затем удаляем.
- Удаление контейнера не равно удалению образа.
- Не удаляйте контейнер, если в нём нужны несохранённые данные.

Шпаргалка по командам

Команда	Действие
docker run	Создать и запустить
docker ps	Показать работающие
docker ps -a	Показать все
docker stop	Остановить
docker start	Запустить существующий
docker restart	Перезапустить
docker rm	Удалить

Разница между docker run и docker start

Обе команды запускают контейнер, но работают по-разному

docker run

Создаёт и запускает новый контейнер

Команда **docker run** скачивает образ (если его нет), создаёт новый контейнер и сразу запускает его.

```
docker run -d --name web -p 8080:80 nginx:stable
```

1. Проверяет наличие образа
2. Скачивает образ (если нет)
3. Создаёт контейнер
4. Запускает контейнер



Образ **nginx:stable** → Новый контейнер **web** → Контейнер работает

Особенности docker run

- Создаёт новый контейнер каждый раз.
- Можно сразу задать параметры: имя, порты, тома, переменные окружения.
- Если образа нет — скачивает его из Docker Hub.
- Используется, когда нужно запустить новое приложение.

docker start

Запускает существующий остановленный контейнер

Команда **docker start** просто запускает уже созданный контейнер. Никаких новых образов не скачивает и новых контейнеров не создаёт.

```
docker start web
```

1. Находит существующий контейнер
2. Запускает контейнер



Остановленный контейнер **web** → Контейнер работает (те же настройки)

Особенности docker start

- Запускает только уже созданный контейнер.
- Не создаёт новый контейнер.
- Использует все ранее заданные настройки (порты, тома и т. д.).
- Удобно для повторного запуска остановленного контейнера.

Когда какую команду использовать?

docker run — когда нужно запустить новое приложение (впервые или с другими настройками).

docker start — когда нужно просто запустить уже созданный и остановленный контейнер.

Частая ошибка: имя уже занято

- Команда run создаёт новый контейнер.
- Если web1 уже существует, вторично создать его с этим именем нельзя.
- Проверьте docker ps -a.
- Используйте docker start web1 либо удалите ненужный старый контейнер.

Режимы работы контейнера

Фон и интерактивная оболочка

Фоновый режим -d

- Подходит для долго работающих сервисов, например Nginx.
- Контейнер работает, терминал свободен.
- Чтобы увидеть его состояние, используем docker ps.

Интерактивная команда

```bash
sudo docker exec -it web1 sh
```

Что важно

- exec запускает дополнительную команду в работающем контейнере.
- -i позволяет вводить команды, -t создаёт терминал.
- sh — простая командная оболочка.

Команды внутри контейнера

```bash
pwd
ls
exit
```

Что важно

- pwd показывает текущий каталог.
- ls показывает файлы.
- exit закрывает оболочку, но не останавливает Nginx.

Осторожно с командами в контейнере

- Не каждый образ содержит bash или nano.
- Для Nginx используем оболочку sh.
- Не требуется устанавливать новые пакеты внутри контейнера.
- Изменения внутри удаляемого контейнера можно потерять.

docker exec: терминал Debian и оболочка внутри контейнера

Терминал Debian

student@debian: ~

```
student@debian:~$ sudo docker \
exec -it web1 sh
```

Команда выполняется
на хосте Debian

Debian 13 (хост)

Ваш сервер
с Debian 13

docker exec

Контейнер web1

Оболочка внутри контейнера

```
/# pwd
/
/# ls
bin  etc  usr  var  ...
/# exit
```

Команды выполняются
внутри контейнера

Nginx

порт 80

Команда

```
sudo docker exec -it web1 sh
```

- `-i` — интерактивный режим
- `-t` — псевдотерминал
- `web1` — имя контейнера
- `sh` — оболочка внутри контейнера

Что делает docker exec

1. Подключает вас к уже работающему контейнеру
2. Позволяет выполнить команду внутри контейнера
3. Не создаёт новый контейнер

Важно

1. web1 должен быть запущен
2. exit закрывает оболочку, но не останавливает Nginx
3. Для Nginx часто используют sh, а не bash

Терминал Debian

```
student@debian:~$
```

После exit

возвращаемся
в терминал Debian

Контейнер web1

Nginx

Nginx
продолжает работать

Проверяем и диагностируем

Логи, состояние и работа Nginx

Посмотреть журналы контейнера

```bash
sudo docker logs web1
```

Что важно

- Показывает сообщения приложения.
- После обращения к сайту Nginx обычно фиксирует HTTP-запрос.

Последние строки журнала

```bash
sudo docker logs --tail 10 web1
```

Что важно

- --tail 10 показывает последние 10 строк.
- Удобно, если сообщений много.

Основные сведения о контейнере

```bash
sudo docker inspect web1
```

Что важно

- Показывает подробные технические параметры.
- В выводе можно найти имя, состояние и параметры портов.
- Для первого знакомства достаточно знать, что команда существует.

Что проверить, если сайт недоступен?

- Контейнер запущен? sudo docker ps
- Адрес введён правильно? http://localhost:8080
- Не занят ли порт 8080 другим приложением?
- Есть ли ошибки в sudo docker logs web1?

Простая диагностика Nginx: контейнер, порт, запрос, журнал

1 Контейнер

Проверьте, запущен ли контейнер **web1**.
Статус должен быть Up.




```
$ sudo docker ps
# sudo docker ps -a
```

2 Порт

Проверьте правильность публикации порта.
Слева порт хоста, справа порт контейнера.




```
$ sudo docker run -d --name web1 \
  -p 127.0.0.1:8080:80 \
  nginx:stable
```

3 Запрос

Отправьте запрос на **http://localhost:8080**.
Если Nginx работает, вернётся HTML-страница.
localhost в Debian — это сама Debian.




```
$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head><title>Welcome to nginx!</title></head>
<body>
<h1>Welcome to nginx!</h1>
..</body></html>
```

4 Журнал

Журнал показывает, дошёл ли запрос до Nginx.
Ищите обращения к **/** и код ответа 200.

```
$ sudo docker logs --tail 10 web1

172.17.0.1 - - [10/Apr/2025:10:15:32 +0000]
"GET / HTTP/1.1" 200 615 "-" "curl/8.6.0"

172.17.0.1 - - [10/Apr/2025:10:16:10 +0000]
"GET /favicon.ico HTTP/1.1" 404 153 "-"
"curl/8.6.0"

172.17.0.1 - - [10/Apr/2025:10:17:05 +0000]
"GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0"
```

Если сайт не открывается

1. Контейнер запущен?
2. Порт указан правильно?
3. Запрос отправляется на localhost:8080?
4. Есть ли записи в docker logs?

Важно

- **docker exec** не нужен для простой проверки сайта.
- **curl** и **docker logs** — самые быстрые инструменты диагностики.
- После **stop** контейнер перестаёт отвечать.

Можно запустить второй Nginx

- Из образа nginx:stable можно создать ещё один контейнер.
- Нужно другое имя, например web2.
- При публикации нужен свободный порт Debian, например 8081.
- Один контейнер не мешает другому слушать порт 80 внутри себя.

Запуск второго Nginx

```bash
sudo docker run -d --name web2 -p 127.0.0.1:8081:80 nginx:stable
```

Что важно

- web2 — другое имя контейнера.
- 8081 — другой порт Debian.
- Внутренний порт Nginx по-прежнему 80.

Проверяем второй сайт

```bash
curl http://localhost:8081
```

Что важно

- Проверяем отдельный контейнер.
- Оба Nginx используют одинаковый образ, но работают независимо.

Главное за сегодня

1. docker run создаёт новый контейнер; docker start запускает существующий.
2. docker ps и docker ps -a показывают состояние контейнеров.
3. docker stop, restart и rm управляют жизненным циклом.
4. docker exec выполняет команду внутри работающего контейнера.
5. Публикация порта даёт доступ к Nginx, docker logs помогает искать ошибки.

Контрольные вопросы

1. Чем образ отличается от контейнера?
2. Что означает -d?
3. Для чего нужен --name?
4. Почему docker run и docker start не одно и то же?
5. Чем отличаются docker ps и docker ps -a?
6. Что означает 127.0.0.1:8080:80?
7. Что делает docker exec -it web1 sh?
8. Зачем нужна команда docker logs?
9. Почему после exit Nginx продолжает работу?
10. Почему двум контейнерам нужны разные опубликованные порты?